Transparent, Post-Quantum, and Sublinear-Space Zero-knowledge Proofs

Logan Nye, MD

Carnegie Mellon University School of Computer Science
5000 Forbes Ave Pittsburgh, PA 15213 USA

lnye@andrew.cmu.edu

ORCID: 0009-0002-9136-045X

Abstract

This manuscript proposes and analyzes an idealized architecture for an "optimal" zero-knowledge proof (ZKP) system: a transparent, plausibly post-quantum, execution-trace-centric proof system with sublinear-space provers for arbitrarily long computations. We articulate a precise desiderata list for long-term, infrastructure-grade ZKPs, then design a STARK-like, hash-based interactive oracle proof (IOP) that integrates a height-compressed, $O(\sqrt{T})$-space prover architecture. The core insight is to treat the prover as a deterministic machine executing over a very long trace and to apply a Height Compression Theorem to reshape the prover's computation DAG into a form that admits pointerless depth-first traversal and algebraic replay of trace segments. This shifts the primary bottleneck from RAM to computation, enabling proofs for programs whose length T greatly exceeds physical memory, while preserving succinct public verification and zero-knowledge. We give a more detailed system architecture, provide step-by-step algorithms and pseudocode for the prover, verifier, and algebraic replay engine, and discuss how the design composes with recursion, zk-VMs, zkML, and verifiable cloud compute. The resulting system is proposed as a blueprint for future-proof zero-knowledge infrastructure.

1 Introduction: Toward a "Perfect" Zero-Knowledge System

1.1 Motivation and Vision

- Zero-knowledge proofs are becoming a foundational primitive for scalable blockchains, privacy-preserving protocols, verifiable cloud compute, and confidential machine learning.
- Today, ZKP ecosystems are fragmented across proof systems (SNARKs, STARKs, variants), each with different trust assumptions, performance envelopes, and security stories.
- Provers for real-world workloads (zkEVMs, zkVMs, zkML, large data pipelines) are predominantly *memory-bound*: they require hundreds of GB of RAM for polynomials, FFT buffers, permutation accumulators, and quotient polynomials.
- This manuscript aims to articulate and formalize an architecture that, while idealized, captures what a "perfect" or *future-proof* ZKP system could look like, with a particular emphasis on:
  1. Transparency and plausibly post-quantum security.
  2. Execution-trace-centric design.
  3. Sublinear-space provers using height compression and algebraic replay.
  4. Clean integration with recursion and proof aggregation.

1.2 Current Pain Points in Modern ZKP Stacks

- **Prover memory bottleneck:** Proving large traces forces $O(T)$ memory for polynomial buffers, FFT/IFFT, and Merkle / FRI layers.
- **Operational brittleness:** Horizontal scaling is difficult; failure modes in distributed proving are subtle and protocol-dependent.
- **Developer constraints:** Program design is constrained by what can be proven within RAM limits rather than what faithfully models the underlying computation.
- **Hardware inefficiency:** GPUs excel at FFT/MSM but are limited by VRAM; large working sets exceed L2/L3 cache, causing cache thrash and poor locality.

1.3 Central Hypothesis

$$\text{There exists a ZKP architecture with transparent, hash-based IOPs and provers running in } O(\sqrt{T}) \text{ space} \tag{1}$$

for computation traces of length T, such that:

1. The proof system is *transparent* (no trusted setup) and plausibly *post-quantum* (hash/-coding assumptions).
2. Verifier cost is polylogarithmic in T and polynomial in the security parameter.
3. Provers handle arbitrarily long deterministic traces with working memory $O(\sqrt{T})$ by using height-compressed computation DAGs and algebraic replay of trace segments.
4. The system supports succinct recursion and aggregation, enabling scalable rollups and verifiable compute.

2 Design Principles for a "Perfect" ZKP System

2.1 Long-Term Desiderata

We specify the properties we view as essential for long-lived ZKP infrastructure:

1. **Transparency:** No trusted setup, no structured reference string.
2. **Plausible post-quantum security:** Relies on hash functions and coding assumptions rather than pairings.
3. **Succinct public verification:** Verifier runtime $O(\text{polylog}(T) \cdot \text{poly}(\lambda))$ and proof size sublinear in T.
4. **Sublinear-space proving:** Prover working memory $o(T)$, with a concrete target of $O(\sqrt{T})$.
5. **Execution-trace focus:** Programming model built around traces of a VM or circuit; arithmetization respects local transition structure.
6. **Recursion and aggregation:** Native support for recursive proofs and batch aggregation.
7. **Programmability:** Support for high-level language compilation, VMs, and ML kernels without extreme manual circuit engineering.
8. **Implementation robustness:** Suitability for real-world distributed deployments, fault-tolerant proving, and robust monitoring.

2.2 STARK-Like, Hash-Based Cores as the Right Foundation

We adopt a STARK-like, IOP-based architecture as the foundation:

- Hash-based commitments for codewords and traces (Merkle trees, vector commitments).
- FRI-style low-degree tests for soundness.
- AIR- or Plonkish-style arithmetization over execution traces.

Pairing-based SNARKs can be integrated for recursion or compression, but the core proving engine is transparent and hash-based.

3 Formal Model: Execution-Trace-Based IOPs

3.1 Execution Trace and Constraints

Let C be a computation we wish to prove, with trace length T and width w.

$$C : (x, w) \mapsto y, \quad \text{with trace } s_0, s_1, \ldots, s_T \in \mathbb{F}^w \tag{2}$$

where s_0 encodes (x, w), s_T encodes y, and for each t:

$$s_{t+1} = F(s_t) \tag{3}$$

for some local transition function F that can be encoded as low-degree constraints.

Definition 1 (Execution-Trace IOP)**.** *An execution-trace IOP is an interactive oracle proof in which the prover's main oracles are low-degree encodings of the trace columns and auxiliary polynomials capturing constraint evaluations. The verifier inspects a small number of random positions in these oracles to check correctness.*

3.2 Prover as a Deterministic Machine

We explicitly model the prover as a deterministic multitape Turing machine M:

- Input: (x, w), protocol description, security parameter λ, randomness seed r for Fiat–Shamir.
- Output: proof π (commitments, query answers, and transcripts).
- Internal state: intermediate polynomials, Merkle roots, FRI layers, and hash inputs.

The computation of M induces a DAG where nodes are intermediate states and edges represent data dependencies. Our goal is to restructure this DAG into a height-compressed form with bounded stack depth and bounded-width state.

4 Height Compression and Sublinear-Space Provers

4.1 Height-Compressible Prover Computations

Definition 2 (Height-Compressible Prover)**.** *A prover computation of length T is height-compressible if its canonical computation tree can be reshaped into a binary tree of height $O(\log T)$ such that:*

1. *Leaf computations only require $O(b)$ working memory for some block size b.*
2. *Internal node merges require $O(1)$ or $O(\log T)$ additional memory.*
3. *A pointerless depth-first traversal evaluates the tree while storing at most $O(\log T)$ stack frames.*

We view the prover's overall work as a sequence of *block-local* computations (on trace segments and polynomial tiles) combined with *associative* merging operations (hashes, linear combinations, FRI folding steps).

4.2 Height Compression Theorem (Informal)

Theorem 1 (Height Compression for Deterministic Provers (Informal))**.** *Let M be a deterministic prover machine that, given a trace of length T, runs in time $\mathrm{TIME}[T \cdot \mathrm{polylog}(T)]$ with local, block-respecting operations over segments of length b. Then there exists an equivalent machine M' that computes the same proof transcript using $O(\sqrt{T})$ working memory via:*

- *A balanced computation tree over $B = \lceil T/b \rceil$ blocks.*
- *Pointerless depth-first traversal with $O(\log B)$ stack frames.*
- *An algebraic replay engine that recomputes block-local results on demand using $O(b)$ memory.*

Choosing the block size

$$b \approx \sqrt{T} \tag{4}$$

yields $B \approx \sqrt{T}$ and working space $O(b + \log B) = O(\sqrt{T})$.

4.3 Algebraic Replay Engine

The algebraic replay engine is a subroutine that, on input a block index i, recomputes all locally necessary objects for that block:

- Row values for the trace segment $s_{ib}, \ldots, s_{(i+1)b-1}$.
- Evaluations of constraint polynomials on that segment.
- Local contributions to codewords used in FRI.
- Merkle leaves corresponding to this segment in each commitment tree.

It operates using $O(b)$ working memory and runs in time $\mathrm{poly}(b, \log T, \lambda)$.

5 Formal System Specification

5.1 Cryptographic Primitives and Global Parameters

- Security parameter λ.
- Finite field $\mathbb{F}$ with $|\mathbb{F}| \geq 2^\lambda$ or appropriate extension $\mathbb{F}_{q^k}$.
- Collision-resistant hash function $H : \{0,1\}^* \to \{0,1\}^\lambda$ used for Merkle trees and Fiat–Shamir.
- FRI parameters: evaluation domain size $N \geq T$, number of rounds r, folding factor, query complexity.

5.2 Execution-Trace Language and Arithmetization

We define a description language $\mathcal{L}$ for computations to be proven:

- $\mathcal{L}$ programs compile to a VM or circuit with a fixed instruction set.
- The VM state at time t is represented by $s_t \in \mathbb{F}^w$.
- Transition constraints are specified as a system of low-degree polynomials over windows of rows, e.g.,

$$Q_j(s_t, s_{t+1}, \ldots, s_{t+\Delta}) = 0 \quad \forall t, j \tag{5}$$

This yields an AIR-like or Plonkish-like constraint system over the trace.

5.3 Commitment and Codeword Layer

- The trace columns and auxiliary polynomials are viewed as evaluations of low-degree polynomials over a domain $D \subseteq \mathbb{F}$.
- For each such polynomial f, the prover commits to its evaluations $f(\alpha)$ for $\alpha \in D$ via a Merkle tree over the vector $(f(\alpha_0), \ldots, f(\alpha_{N-1}))$.
- All Merkle trees are constructed using the same hash function H with a canonical layout.

5.4 FRI-Based Low-Degree Testing

- Given a codeword $f|_D$, FRI constructs a sequence of folded codewords with decreasing length, each committed via Merkle trees.
- The verifier samples random indices and checks consistency across layers, reducing the probability that f is far from a low-degree polynomial.

6 Algorithms and Pseudocode

In this section, we give explicit pseudocode for:

1. System configuration (global parameters, arithmetization).
2. The height-compressed prover algorithm.
3. The algebraic replay engine for block-local computations.
4. The verifier algorithm.
5. A recursive wrapper (high-level sketch).

6.1 System Configuration Algorithm

Algorithm 1 CONFIGURE-SYSTEM(λ)

```
Choose finite field 𝔽 with |𝔽| ≥ 2^λ.
Fix collision-resistant hash function H : {0,1}* → {0,1}^λ.
Select evaluation domain D ⊆ 𝔽 with |D| = N such that N ≥ T_max.
Fix FRI parameters: number of rounds r, query complexity q, folding factor, etc.
Specify VM / circuit model and its AIR/Plonkish arithmetization.
Publish global parameters pp = (𝔽, H, D, r, q, VM, AIR).

return pp.
```

6.2 Height-Compressed Prover Algorithm (Top-Level)

We describe the prover as a block-wise, height-compressed engine over the trace.
The subroutines above are all implemented via the algebraic replay engine and height-compressed traversal. We now spell out these components in more detail.

6.3 Height-Compressed Commitments

We consider first the problem of committing to a large codeword $v \in \mathbb{F}^N$ (e.g., evaluations of a trace column) using a Merkle tree, but with $O(\sqrt{T})$ space.

We assume that v is implicitly defined via the trace and arithmetization and can be generated block-by-block using a replay engine.
This pointerless DFS strategy never stores more than $O(\log L)$ hashes plus the current block-local values from $G(i)$, yielding $O(\sqrt{T})$ memory when $b \approx \sqrt{T}$.

6.4 Algebraic Replay Engine

The replay engine is responsible for block-local re-derivation of trace and polynomial evaluations. In practice, the replay strategy may use intermediate checkpoints to avoid $O(T)$ recomputation, but the working memory remains $O(b)$.

Algorithm 2 PROVE-HEIGHT-COMPRESSED(pp, C, x, w)

Require: Global parameters pp, computation description C, public input x, witness w.
Ensure: Proof π attesting that $C(x, w)$ executes correctly.

```
(trace, T) ← EXECUTE-VM(C, x, w) {Generate execution trace}
Choose block size b ← ⌈√T⌉ and number of blocks B ← ⌈T/b⌉
Sample Fiat–Shamir seed r_FS ← H(C, x, T)
Initialize transcript 𝒯 ← ∅
/* Phase 1: Commit to trace and auxiliary polynomials */
𝒞 ← BUILD-HEIGHT-COMPRESSED-COMMITMENTS(pp, trace, b, B, r_FS, 𝒯)
Append 𝒞 to transcript 𝒯
Derive first verifier challenge α_1 ← H(𝒯)
/* Phase 2: Construct FRI codewords and commitments (height-compressed) */
ℱ ← BUILD-FRI-LAYERS-HEIGHT-COMPRESSED(pp, trace, b, B, α_1, r_FS, 𝒯)
Append ℱ to transcript 𝒯
Derive full set of verifier challenges (α_2, ..., α_r) ← H(𝒯)
/* Phase 3: Answer verifier queries with block-local replay */
𝒬 ← ANSWER-QUERIES-WITH-REPLAY(pp, trace, b, B, (α_i)_i, r_FS, 𝒯)
π ← (𝒞, ℱ, 𝒬)

return π
```

Algorithm 3 BUILD-HEIGHT-COMPRESSED-COMMITMENTS($\mathsf{pp}, \mathrm{trace}, b, B, r_{\mathsf{FS}}, \mathcal{T}$)

```
Initialize empty list of commitments 𝒞
for each polynomial family P ∈ {trace columns, aux polynomials} do
    Define block-level generator G_P(i) that yields evaluations of P on block i using replay
    Let N_P be total evaluation length (aligned to B · b)
    Define implicit Merkle tree ℳ_P with N_P leaves
    Compute Merkle root root_P via pointerless DFS:
        root_P ← MERKLE-ROOT-HEIGHT-COMPRESSED(G_P, B, b)
    Append (P, root_P) to 𝒞
end for

return 𝒞
```

6.5 Height-Compressed FRI Layer Construction

FRI codewords can be built layer-by-layer using the replay engine and height-compressed Merkle constructions.

6.6 Answering Verifier Queries via Replay

Finally, we answer the verifier's random queries by replaying only the necessary blocks. Here, MERKLE-PATH-HEIGHT-COMPRESSED mirrors MERKLE-ROOT-HEIGHT-COMPRESSED but records the sibling hashes along a single root-to-leaf path, using on-the-fly recomputation instead of storing the full tree.

6.7 Verifier Algorithm

The verifier is standard for a STARK-like IOP, with the only difference that the prover may have used height compression internally. The verifier is oblivious to that internal structure.

Algorithm 4 MERKLE-ROOT-HEIGHT-COMPRESSED(G, B, b)

Require: $G(i)$ yields the values for leaves corresponding to block i, $i \in \{0, \ldots, B-1\}$
Ensure: Merkle root of the full codeword

```
Let L = B · b be number of leaves (pad as needed)
Conceptually define a complete binary tree over L leaves; do not materialize it
Initialize explicit stack S ← ∅
Initialize leaf index ℓ ← 0
while true do
  if we are at a leaf position corresponding to index ℓ then
    Determine block index i ← ⌊ℓ/b⌋
    blockVals ← G(i) {Replay block i; uses O(b) memory}
    Extract appropriate entry v_ℓ from blockVals
    Compute leaf hash h_ℓ ← H(v_ℓ)
    Push (h_ℓ, depth = 0) onto stack S
    ℓ ← ℓ + 1
  else
    Let (h_1, d_1) and (h_2, d_2) be the top-most elements of S
    if d_1 = d_2 then
      Pop (h_1, d_1) and (h_2, d_2) from S
      h_parent ← H(h_1 ‖ h_2)
      Push (h_parent, d_1 + 1) onto S
    else
      if ℓ = L and |S| = 1 then

        return the unique hash in S as the Merkle root
      else
        Move to the next leaf (if any) or combine nodes when possible
      end if
    end if
  end if
end while
```

6.8 Recursive Wrapper (High-Level Sketch)

To support recursion, we embed the verification logic above into an arithmetic circuit or VM that itself can be arithmetized and proven in the same STARK-like framework or with a SNARK overlay.

7 System Architecture and Implementation Blueprint

7.1 Logical Architecture

The system is structured into layers:

1. **Frontend / DSL:** High-level language or API used by developers to specify computations C.

2. **Compiler / VM:** Compiles C into VM instructions and generates the execution trace.

3. **Arithmetization:** Converts VM transitions into AIR/Plonkish constraints over the trace.

Algorithm 5 REPLAY-BLOCK($\mathsf{pp}, C, x, w, i, b$)

Require: Block index i, block size b, global parameters pp, computation C, inputs (x, w).
Ensure: Local block trace and polynomial evaluations for rows $t \in [ib, (i+1)b)$.

```
Determine time window [t_min, t_max) = [ib, min((i+1)b, T))
Re-initialize VM state to known checkpoint before t_min, or recompute from t = 0
for t = t_min to t_max − 1 do
    Compute next state s_{t+1} ← F(s_t)
    Store s_t in a local buffer blockTrace (size at most b)
end for
Evaluate constraint polynomials on blockTrace as needed:
for each constraint Q_j do
    for each t in local window where Q_j applies do
        Compute Q_j(s_t, s_{t+1}, ...) and add to local buffers
    end for
end for
Convert local trace and constraints into evaluations of trace/aux polynomials on corresponding domain points; store in blockPolys

return (blockTrace, blockPolys)
```

Algorithm 6 BUILD-FRI-LAYERS-HEIGHT-COMPRESSED($\mathsf{pp}, \mathrm{trace}, b, B, \alpha_1, r_{\mathsf{FS}}, \mathcal{T}$)

```
Initialize list F ← ∅
Let f^(0) be the initial composition polynomial (implicitly defined over D)
for ℓ = 0 to r − 1 do
    Define block-level generator G^(ℓ)(i) using REPLAY-BLOCK and prior layer info
    root^(ℓ) ← MERKLE-ROOT-HEIGHT-COMPRESSED(G^(ℓ), B_ℓ, b_ℓ)
    Append root^(ℓ) to F
    Derive next folding challenge β_ℓ ← H(T, root^(ℓ))
    Define next layer f^(ℓ+1) via FRI folding with challenge β_ℓ
end for

return F
```

4. **Algebraic Replay Engine:** Provides block-local regeneration of trace and polynomial evaluations.

5. **Height-Compressed Commitment Layer:** Builds Merkle trees and FRI codewords using pointerless DFS and replay.

6. **IOP Orchestrator:** Coordinates Fiat–Shamir challenges, commitments, and queries.

7. **Verifier Library:** Implements verification logic, accessible to applications and chains.

7.2 Physical Architecture and Hardware Mapping

- **Single-node deployment:** Run the height-compressed prover on a single machine with modest RAM; extra cores/GPUs primarily increase throughput by parallelizing block-level replay.

- **Distributed deployment:** Assign disjoint block ranges to different workers, each running REPLAY-BLOCK and transmitting local commitments/roots; aggregate via height-compressed merging.

Algorithm 7 ANSWER-QUERIES-WITH-REPLAY($\mathsf{pp}, \text{trace}, b, B, (\alpha_i)_i, r_{\mathsf{FS}}, \mathcal{T}$)

Derive query positions $\mathcal{I} \subseteq \{0, \ldots, N-1\}$ from transcript and challenges
Initialize answer set $\mathcal{Q} \leftarrow \emptyset$
for each index $j \in \mathcal{I}$ **do**
  Determine block index $i \leftarrow \lfloor j/b \rfloor$
  $(\texttt{blockTrace}, \texttt{blockPolys}) \leftarrow$ REPLAY-BLOCK($\mathsf{pp}, C, x, w, i, b$)
  Extract required evaluations (trace, aux polynomials, FRI layers) for index j
  Construct Merkle authentication paths for all relevant trees at index j using MERKLE-PATH-HEIGHT-COMPRESSED
  Append evaluation values and Merkle paths for j to $\mathcal{Q}$
end for

return $\mathcal{Q}$

Algorithm 8 VERIFY-HEIGHT-COMPRESSED($\mathsf{pp}, C, x, y, \pi$)

Require: Global parameters pp, computation description C, input/output (x, y), proof $\pi = (\mathcal{C}, \mathcal{F}, \mathcal{Q})$.
Ensure: Accept or reject the statement: "There exists a witness w such that $C(x, w) = y$."

Parse commitments $\mathcal{C}$ and FRI roots $\mathcal{F}$.
Recompute Fiat–Shamir challenges $(\alpha_1, \ldots, \alpha_r)$ from $(C, x, y, \mathcal{C}, \mathcal{F})$.
Derive query indices $\mathcal{I}$ from $(\alpha_i)_i$.
For each $j \in \mathcal{I}$:
  Verify Merkle authentication paths for all trace and aux polynomial commitments at index j.
  Check local transition constraints and boundary conditions for the retrieved rows.
  Verify FRI consistency checks across layers at index j.
If all checks pass, **return** Accept; else **return** Reject.

- **Edge deployment:** For smaller zkML or private analytics workloads, run on edge devices where T is large relative to RAM, but $\sqrt{T}$ fits comfortably in cache or DRAM.

8 Applications and Case Studies

8.1 Rollup Provers and zkVMs

- Long-running rollup traces: T proportional to total L2 activity over many blocks.
- Height-compressed proving allows these traces to be proven on machines with memory footprint $\tilde{O}(\sqrt{T})$, decoupling throughput from RAM.

8.2 zkML and Large-Scale Private Inference

- Deep networks correspond to long trace sequences; parameters and activations are streamed.
- Replay-based proving allows working sets (e.g., layer activations) to fit into VRAM or L2/L3 cache.

8.3 Verifiable Cloud Compute

- Arbitrary compute pipelines (ETL, analytics, scientific simulations) can be modeled as traces.

Algorithm 9 RECURSIVE-PROVE($\mathsf{pp}, \{(C_k, x_k, y_k)\}_k$)

For each instance (C_k, x_k, y_k):
    $\pi_k \leftarrow$ PROVE-HEIGHT-COMPRESSED($\mathsf{pp}, C_k, x_k, w_k$)
Build a higher-level circuit/VM $\mathcal{V}$ that:
    Hard-codes pp and all (C_k, x_k, y_k).
    Executes the verification algorithm VERIFY-HEIGHT-COMPRESSED for each (C_k, x_k, y_k, π_k).
Compile $\mathcal{V}$ into a new trace and apply PROVE-HEIGHT-COMPRESSED to obtain recursive proof Π.

return Π as an aggregate proof for all instances.

- Height-compressed ZKPs certify correctness while hiding inputs, suitable for cloud verification.

9 Related Work

- Pairing-based SNARKs: Groth16, Plonk, Marlin, and variants.
- Transparent STARKs and FRI-based IOPs.
- Streaming polynomial commitments and memory-efficient FFTs.
- Recursive proof systems and aggregation frameworks.

10 Discussion and Future Directions

10.1 Open Problems

- Formalizing general conditions for height-compressibility of arbitrary prover pipelines.
- Quantifying tradeoffs between recomputation overhead and cache locality in realistic hardware models.
- Designing fully transparent recursive schemes with comparable efficiency to pairing-based SNARK recursion.
- Extending the model to probabilistic and non-deterministic computations while retaining sublinear space.

10.2 The Perfect ZKP System as a Blueprint

- The combination of:
  - Transparent, hash-based IOPs,
  - Execution-trace-centric arithmetization,
  - Height-compressed $O(\sqrt{T})$-space provers, and
  - Native recursion and aggregation,
- offers a concrete blueprint for future-proof, long-lived zero-knowledge infrastructure.
- Implementations approximating this ideal can push ZK from a niche cryptographic tool into a foundational layer of computation and data infrastructure.